PEC 2

Clases y objetos

Universitat Oberta de Catalunya

Información relevante:

- Fecha límite de entrega: 26/10/2026.
- Peso en la nota de PECs: 20%.

Contenido

Información docente **3**
Prerrequisitos 3
Objetivos 3
Resultados de aprendizaje 3
Enunciado **4**
Tarea 1 - Proyecto Gradle 4
Creación de un proyecto Gradle en IntelliJ 5
Testing con JUnit + Gradle 9
Ejercicio 1 (5 puntos) 12
Ejercicio 2 (1 punto) 19
Ejercicio 3 (1 punto) 20
Ejercicio 4 (3 puntos) 22
Formato y fecha de entrega **25**
Anexo: Problemas frecuentes **26**
Espacios y caracteres especiales en la ruta del proyecto 26
El IDE no encuentra ningún test para ejecutar 26
Limpiar el proyecto elimina el directorio javadoc 27



En esta actividad no está permitido el uso de herramientas de inteligencia artificial.

En el plan docente y en la web sobre integridad académica y plagio de la UOC encontraréis información sobre qué se considera conducta irregular en la evaluación y las consecuencias que puede tener.

Información docente

Esta PEC está vinculada al módulo teórico "Abstracción y encapsulación" de los apuntes de la asignatura. Léelo antes de empezar la PEC.

Prerrequisitos

Para realizar esta PEC necesitas:

- Tener adquiridos los conocimientos de la PEC 1. Por eso, te recomendamos que mires la solución que se publicó en el aula y la compares con la tuya.

Objetivos

Con esta PEC, el equipo docente de la asignatura busca que:

- Sepas diferenciar entre objeto y clase, siendo capaz de abstraer la clase a partir de un conjunto de objetos/entidades reales que están relacionados entre sí.
- Veas el potencial de la encapsulación.
- Utilices Java como un lenguaje de programación orientado a objetos.
- Crees programas bien documentados que faciliten su mantenimiento posterior.
- Entiendas qué es una excepción y cómo se utiliza en Java.
- Conozcas las diferencias entre las clases `String`, `StringBuilder` y `StringBuffer`.
- Acabes de familiarizarte con la dinámica de la asignatura.

Resultados de aprendizaje

Con esta PEC debes demostrar que eres capaz de:

- Codificar una clase a partir de unos requisitos y su representación como diagrama de clases.
- Entender la diferencia entre los diferentes modificadores de acceso.
- Comprender qué consecuencias tiene declarar un miembro como `static`.
- Gestionar casos anómalos, incorrectos e indeseados mediante excepciones.
- Documentar con Javadoc un programa.
- Saber utilizar las clases `String`, `StringBuilder` y `StringBuffer`.
- Utilizar test unitarios para determinar que un programa es correcto.
- Usar con cierta soltura un entorno de desarrollo integrado (IDE) como IntelliJ.

Enunciado

Esta PEC contiene 4 ejercicios evaluables y 1 tarea no evaluable. Debes entregar tu solución de los 4 ejercicios evaluables (ver el último apartado).



Debido a que las actividades están encadenadas (i.e. para hacer una se debe haber comprendido la anterior), **es altamente recomendable hacer las tareas y ejercicios en el orden en que aparecen en este enunciado.**

Tarea 1 - Proyecto Gradle

Antes de empezar, debes:

Leer los apartados del 3.1 al 3.6 de la Guía de Java y ver los 3 primeros vídeotutoriales del apartado **Java** del bloque **UOCoders** que encontrarás en "Contenidos" del aula.

En la PEC anterior vimos cómo crear un proyecto Java con IntelliJ y cómo añadirle la librería de JUnit 5. En esta PEC vamos a entrar en el mundo de la "automatización de *builds*" (construcciones) y la gestión de dependencias, y lo haremos a través de la herramienta Gradle, la cual nos permite automatizar la construcción de nuestro código. Nos limitaremos a entender y trabajar con ciertas tareas de Gradle, puesto que esta herramienta expone un gran volumen de funcionalidades asociadas a la automatización del desarrollo de proyectos (i.e., compilación, depuración, testing, empaquetado, distribución y despliegue o *deployment)*.

Cuando hablamos de la gestión de dependencias nos referimos a lo que has hecho en la PEC1 cuando añadías manualmente la librería JUnit. Esto en la actualidad no es una práctica habitual y, en su lugar, se utiliza siempre un gestor (Gradle, Maven, etc.) para construir un proyecto (normalmente llamado *build*) y gestionar sus dependencias. Se puede decir que actualmente ya no podemos vivir sin la ayuda de estas herramientas en proyectos profesionales.

Por ejemplo, antes, si una persona entraba a trabajar en una empresa, ésta se descargaba el proyecto Java en su IDE, y tenía que importar todas las dependencias a mano (como hemos hecho en la PEC1), y esto suponía perder una cantidad nada despreciable de tiempo en configurar el entorno de trabajo antes de poder empezar a trabajar realmente con el código de la aplicación. En la actualidad, al trabajar con herramientas del estilo de Gradle, el nuevo programador tan sólo descargará el proyecto, ejecutará la tarea de compilación, y Gradle ya se encargará de resolver y descargar todas las dependencias necesarias (previamente indicadas en el fichero de configuración de Gradle) para que el proyecto funcione correctamente. En cuestión de segundos, tenemos el entorno montado y listo para empezar a desarrollar, sin arduas tareas de configuración. Piensa que a menudo una dependencia (i.e., una librería) de nuestro proyecto puede depender de otra librería que también hay que incluir. Gradle (o Maven, etc.) también se encarga de realizar esta tarea de rastreo y descarga de dependencias de dependencias.

Como características principales de Gradle podemos destacar:

- Flexibilidad: soporta varios lenguajes de programación, a diferencia de Maven que es solo para aplicaciones Java.
- Uso de DSL (*Domain Specific Language*): proporciona un lenguaje sencillo y claro a la hora de construir el fichero de configuración, a diferencia de Maven que está basado en XML.
- Gestión del ciclo de vida: añade la capacidad de soportar todo el proceso de vida del *software* (desde la compilación, pruebas, análisis estadístico e implementación).
- Sistema de gestión de dependencias muy estable.
- Basado en "tareas" (en inglés, *tasks*).

Aquí hay una serie de videos cortos tipo "tutoriales" sobre el funcionamiento de Gradle: https://www.makigas.es/series/automatizando-con-gradle

Como hemos comentado, no nos vamos a centrar en hacer un curso de Gradle, y vamos a ir directamente a cómo crear un proyecto Gradle desde el IDE IntelliJ IDEA.

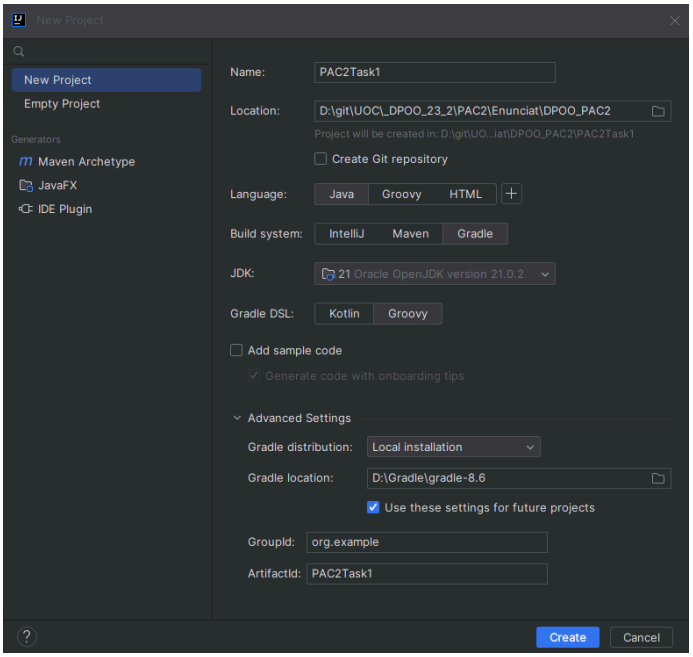
Creación de un proyecto Gradle en IntelliJ

Crea un nuevo proyecto con el botón `New Project` de la pantalla de bienvenida de IntelliJ o desde `File→New→Project` una vez abierto IntelliJ. Una vez clicado el botón para crear un nuevo proyecto, IntelliJ te mostrará una ventana de configuración del proyecto (como la que se muestra en la imagen siguiente). Introduce el nombre del proyecto, en este caso, **`PAC2Task1`**. Verás que ya te selecciona automáticamente tu carpeta *workspace* donde tienes el resto de proyectos (i.e. `IdeaProjects`). Por defecto, IntelliJ selecciona como `Build system` la opción `IntelliJ`. Ahora debemos cambiar y seleccionar `Gradle`, y como lenguaje de programación seleccionamos `Java` (seleccionado por defecto). Haz click en `Create` para crear el proyecto Gradle-Java.



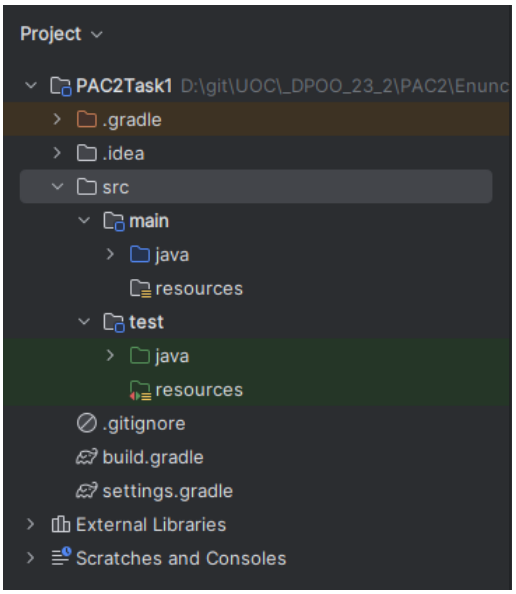
Nota: Es probable que os aparezca un mensaje de tipo *warning* indicando que la versión JDK 21 necesita una versión de Gradle superior que la que viene por defecto. En ese caso, si lo creéis necesario, podéis descargar la versión de **Gradle 8.8** o superior mediante la siguiente página: https://gradle.org/releases/.

Después de la instalación, será necesario seleccionarlo en IntelliJ como instalación local igual que aparece en la siguiente captura de pantalla.



Una vez creado, verás que, en la parte inferior derecha, IntelliJ está ejecutando tareas: descargando el código de Gradle e integrándolo en el proyecto, así como configurando el proyecto Gradle.

Tras configurar y crear el proyecto **`PAC2Task1`**, puedes ver en el *project tool window (*lateral izquierdo) que la estructura o árbol de directorios de un proyecto Gradle (ver siguiente imagen) difiere de la del proyecto Java estándar creado en la PEC anterior.



En él verás la carpeta `src`, la cual ahora contiene dos subcarpetas: (1) `main`, en cuya subcarpeta `main/java` será donde pondrás el código fuente de tu programa, y (2) `test`, donde en la subcarpeta `test/java` será donde pondrás los test unitarios JUnit. Fíjate que

ahora cada carpeta ya tiene asignado el color correspondiente a `Sources Root` (azul) y `Test Sources Root` (verde). Esta manera de estructurar un proyecto es heredada de Maven. De hecho, Gradle no obliga ni crea esta estructura, es IntelliJ quien la crea. Maven ha sido (y sigue siendo) tan utilizado por tantos profesionales que su estructura se ha convertido en un estándar *de facto* (por eso IntelliJ la adopta para los proyectos Gradle).

Por otro lado, verás una carpeta de Gradle, donde está el código de esta herramienta, así como el fichero `gradlew` (versión Windows -`.bat`- y Linux/MacOS) que es el ejecutable de Gradle y el fichero de configuración de Gradle `build.gradle` (con el icono de un elefante).

El fichero `build.gradle` (se podría llamar diferente, pero éste es el nombre que se utiliza por convenio, así pues, es mejor respetarlo) es el que vamos a utilizar para configurar y gestionar los *builds* (i.e. proyectos) y sus dependencias. Analicémoslo brevemente:



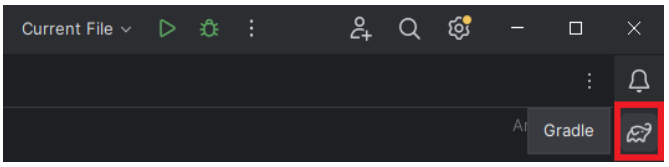
- `plugins`: con el valor `id 'java'` está indicando que estamos utilizando Java, lo que hace que añada tareas específicas de la JVM (compilación, compilación de test, generación de JAR, *clean*, etc.)
- `group`: es el espacio de nombres (*namespace*) para identificar el artefacto o los artefactos producidos por una construcción (*build*). Suele ser el dominio de la empresa en la que trabajamos, p.ej. `edu.uoc`. Su valor puede indicarse al crear el proyecto si se despliega la opción `Artifact Coordinates`.
- `version`: versión del artefacto. También puede indicarse mediante `Artifact Coordinates`.
- `repositories`: por defecto se pone el valor `mavenCentral()`. Este valor indica que las dependencias se obtienen del repositorio central de Maven 2. Puedes ver las librerías incluidas en Maven usando el siguiente buscador: https://search.maven.org/.

- `dependencies`: en este apartado se añaden las librerías que el proyecto necesita para funcionar. En este caso se ha añadido únicamente la librería de test JUnit.
- `test`: con valor `useJUnitPlatform()`. Con el valor anterior se habilita JUnit Platform para ejecutar los test (pruebas). Podríamos tener varias librerías de test descargadas como dependencias y aquí diríamos cuál utilizar.

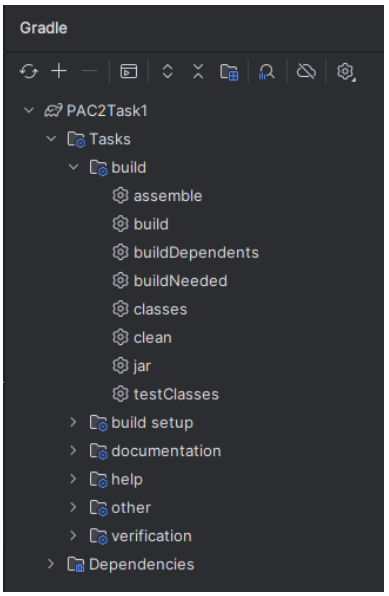


Nota: En la asignatura nos centraremos en las dependencias y en ejecutar las tareas (*task*) de *build* cuando añadamos nuevas dependencias al proyecto. Pero es interesante conocer mínimamente el porqué del resto de entradas del fichero de configuración.

Por último, ahora desde IntelliJ, al tener un proyecto Gradle, te aparece una nueva pestaña para poder ejecutar las diferentes tareas de Gradle (no hace falta ejecutar Gradle mediante línea de comandos, ya que lo tenemos integrado en IntelliJ).



Al desplegar esta pestaña, puedes ver las tareas Gradle que puedes realizar (ver siguiente imagen). Las tareas disponibles dependen, entre otras cosas, de los *plugins* que se indiquen en el fichero `build.gradle`.



Ya tenemos configurado un proyecto con Gradle listo para empezar a trabajar. Aquí te dejamos un vídeo-tutorial oficial de JetBrains sobre Gradle+IntelliJ que explica más detalladamente lo que acabamos de hacer en esta guía: https://youtu.be/6V6G3RyxEMk.

Testing con JUnit + Gradle

Vamos a ver ahora cómo ejecutar JUnit desde Gradle. Evidentemente, como estamos usando un IDE, podemos ejecutar los JUnit como hemos hecho en la PEC1. Pero ya que hemos añadido Gradle, vamos a ver un ejemplo de cómo ejecutar la tarea de `test` para lanzar nuestros test unitarios (y así no centrarnos únicamente en la gestión de dependencias). Para ello, sigue los pasos que se detallan a continuación:

1. En el directorio `src/main/java`, crea el *package* **`edu.uoc.pac2`** donde codificarás tus clases Java. Para ello haz botón derecho sobre el directorio `java` y escoge `New→Package`.

2. En el *package* que has creado, crea la clase **`PAC2Task1`** (botón derecho sobre el *package* y `New→Java Class`) donde tendrás que copiar el siguiente código:

```
package edu.uoc.pac2;

public class PAC2Task1 {
    public static int product(int a, int b){
        return a * b;
    }
}
```

3. Una vez tienes la implementación anterior realizada, vamos ahora a crear la clase que hará de *test case* y que incluirá los *unit tests* de la clase **`PAC2Task1`**. En el directorio `src/test/java`, crea el *package* **`edu.uoc.pac2`** donde tendremos nuestras clases Java de test.

4. En el *package* que acabas de crear, crea la clase **`PAC2Task1Test`** donde testearás la implementación del método `product` de la clase `PAC2Task1`. Copia el siguiente código en el fichero `PAC2Task1Test.java`:

```
package edu.uoc.pac2;

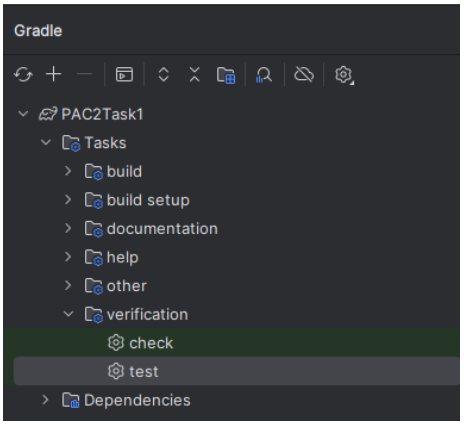
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class PAC2Task1Test {

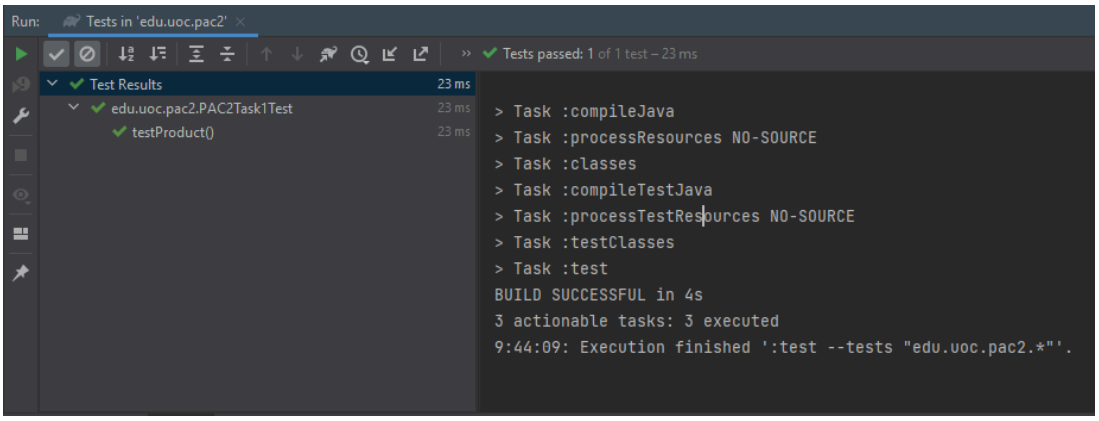
    @Test
    public void testProduct() {
        assertEquals(42, PAC2Task1.product(6, 7));
    }
}
```

5. Ahora ya puedes ejecutar el test implementado en la clase **`PAC2Task1Test`**. Ves a la lista de tareas de Gradle (lateral derecho) y haz doble click en `Tasks→verification→test`, e IntelliJ ejecutará mediante la tarea Gradle los test unitarios (en este caso, tanto la tarea `test` como `check` hacen lo mismo; `Check` ejecuta todos los test y `test` únicamente los test unitarios). Como hemos comentado, puedes ejecutarlo como hemos explicado anteriormente en la PEC1, clicando en el botón derecho sobre la clase y accediendo a `Run`, o con los botones  que verás delante de la clase y los métodos que implementan los test unitarios. Pero de esta manera ya no estarías utilizando Gradle, estarías utilizando el IDE.

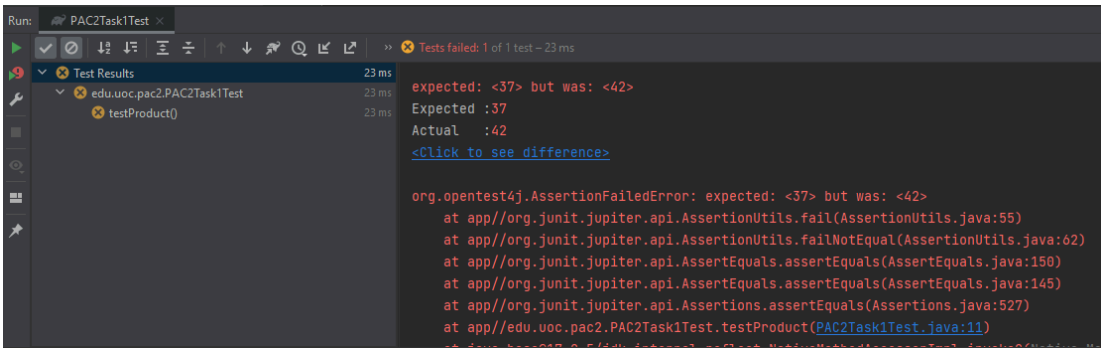


6. En la parte inferior del IDE, puedes ver el resultado de los test. Si pulsas en el botón con forma de *tick* (al lado del *play* verde), entonces se muestran todos los test que han sido ejecutados. Hay 2 posibles estados:

- **Test superado**: muestra un *tick verde* conforme el test ha sido superado satisfactoriamente.



- **Test no superado**: si modificas el valor esperado del test, por ejemplo, en vez de un `42`, pones un `37`, se mostrará que el test no ha sido superado junto con el detalle del error en la parte derecha (traza de error, valor esperado y valor actual del test).



Ejercicio 1 (5 puntos)

Antes de empezar, debes:

> Leer el apartado 4.1 de la Guía de Java que habla sobre la clase `String`.

Abre el proyecto `PAC2Ex1` desde IntelliJ. En el *package* `edu.uoc.pac2` del directorio `/src/test/java` está el fichero con los test unitarios que te proporcionamos. En el *package* `edu.uoc.pac2` del directorio `/src/main/java` tienes que crear y codificar la clase `MuseumArtifact`, cuya representación en diagrama de clases UML te la adjuntamos como imagen dentro del fichero zip del enunciado.

`MuseumArtifact` representa una pieza conservada por un museo. El museo debe almacenar su información básica, controlar su estado de conservación y determinar si puede participar en préstamos temporales solicitados por otras instituciones.

> Todos los nombres de clases, atributos, constantes, variables y métodos, así como los mensajes de error, deben escribirse en inglés.
> Los mensajes de error deben mostrarse mediante `System.out.println`, respetando exactamente el texto indicado, incluido el prefijo [ERROR].

Para codificar la clase `MuseumArtifact`, debes tener en cuenta las siguientes especificaciones y consideraciones:

- Las constantes deben inicializarse en su declaración con los valores deducidos de los requisitos del enunciado. Los atributos de instancia deben asignarse mediante los setters correspondientes desde el constructor.



> **Nota:** Al codificar esta clase según el UML, aparecerán *warnings* (advertencias) indicando que algunos atributos deberían ser `final`. En esta PEC no se explica este modificador, por lo que debes ignorar las advertencias para superar los tests.

- Los métodos *getter* simplemente devuelven el valor del atributo correspondiente.
- La visibilidad de los métodos queda a tu criterio, siempre que puedan utilizarse desde los tests proporcionados. Valora qué métodos forman parte de la interfaz de la clase y cuáles son auxiliares internos.

- **Siempre que haya conflicto entre nombres**, sobre todo en los métodos *setter* (es decir, `setXXX`), **debes usar la palabra reservada `this`**. Si te fijas, el nombre del parámetro de todos los métodos *setters* se llama exactamente como el atributo de instancia que modifican. Si, por ejemplo, hacemos esto:

```
public void setTitle(String title){
   title = title;
}
```

¿Cómo sabe el compilador (y nosotros) si el `title` de la izquierda de la asignación es el atributo de clase (o instancia/objeto) o es el parámetro del método `setTitle`? Y lo mismo ocurre con el `title` del lado derecho. Así pues, para evitar problemas, muchos programadores llaman al parámetro del método utilizando el nombre del atributo a modificar y añadiéndole una "cola", p.ej. `New`:

```
private void setTitle(String titleNew){
 title = titleNew;
}
```

Con esta solución no cabe duda, ni por el compilador ni por nosotros, de qué es qué. Aunque podemos utilizar la solución anterior, lo habitual (y mejor) es utilizar como nombres de los parámetros de los métodos exactamente los mismos que los de los atributos de la clase. Para resolver el conflicto de qué es qué, se utiliza la palabra reservada `this`:

```
private void setTitle(String title){
 this.title = title;
}
```

La palabra reservada `this` hace referencia al objeto (o instancia) que es del tipo de clase que estamos codificando. Por tanto, gracias a `this`, estamos diciendo que en la asignación el `title` de la izquierda es del objeto/clase y el `title` de la derecha es el parámetro del método `setTitle`. Podemos decir que el `this` hace de añadido e indica que lo que lleva `this` pertenece al objeto/clase y no es ni un parámetro ni una variable declarados dentro del método.

Por ejemplo, si tenemos un objeto de tipo `MuseumArtifact` asignado a la variable `artifact`, entonces cuando llamamos a `setTitle` haciendo `artifact.setTitle("Lady with a Fan")`, el `this` que utilizamos dentro del método `setTitle` se refiere a este objeto `artifact`.

- Los atributos y constantes deben declararse con el nombre, tipo y visibilidad indicados en el diagrama UML. Los valores de las constantes deben deducirse de los requisitos del enunciado.

- Las categorías válidas son `PAINTING`, `SCULPTURE`, `MANUSCRIPT`, `TEXTILE`, `CERAMIC` y `OTHER`.

- Los estados de conservación válidos son `EXCELLENT`, `GOOD`, `FAIR` y `POOR`.

- Una pieza se considera de alto valor a partir de `100.000` €. El intervalo entre inspecciones es de `12` meses y la duración de un préstamo debe estar comprendida entre `1` y `365` días, ambos incluidos.

El constructor recibe como parámetros todos los atributos de instancia en el orden mostrado en el diagrama UML. Para asignarlos, debe llamar a los métodos *setter* correspondientes, respetando el mismo orden. Los métodos *getter* simplemente devuelven el valor del atributo correspondiente.

Siempre que el nombre de un parámetro coincida con el de un atributo de instancia, especialmente en los *setters*, debes utilizar la palabra reservada `this` para distinguirlos.

- El método **`setTitle`** asigna el título o denominación de la pieza. El parámetro no puede ser `null`, estar vacío ni contener únicamente espacios en blanco. El valor debe almacenarse sin espacios iniciales ni finales. Si no es válido, el método debe mostrar: `"[ERROR] Title cannot be null, empty or contain only whitespace."`

- El método **`setInventoryCode`** asigna el código de inventario. El valor no puede ser `null` y debe seguir exactamente el formato `ART-YYYY-NNNN`, donde cada `Y` y cada `N` representan un dígito. Si el valor no es válido, el método no debe modificar el atributo y debe mostrar: `"[ERROR] Inventory code must follow the format ART-YYYY-NNNN."`.



Pista: Investiga los métodos de la clase `String`. Para poder detectar si un identificador cumple las restricciones, te recomendamos utilizar una **expresión regular**. Puedes encontrar más información sobre expresiones regulares en:

https://www.javatpoint.com/java-regex

También te puede ser útil utilizar estos dos testeros:

https://regex101.com/
https://www.freeformatter.com/java-regex-tester.html#before-output

O también, utilizar el tester que IntelliJ lleva incorporado: pulsa `Alt+Enter` cuando el cursor del teclado esté en una expresión regular y aparecerá una ventana flotante.

- El método **`setCreator`** asigna el nombre de la persona, colectivo o cultura que creó la pieza. Debe aplicar las mismas reglas de validación y limpieza que `setTitle`. Si el valor no es válido, debe mostrar: `"[ERROR] Creator cannot be null, empty or contain only whitespace."`.

- El método **`setOriginCountry`** asigna el país de origen. El parámetro debe contener exactamente dos letras y debe almacenarse en mayúsculas. Si no es válido, debe mostrar: `"[ERROR] Origin country must be a two-letter code."`.

- El método **`isValidCategory`** devuelve `true` si el parámetro recibido coincide con alguna categoría válida, ignorando mayúsculas y minúsculas. En cualquier otro caso, devuelve `false`.

- El método **`setCategory`** asigna la categoría en mayúsculas si es válida. En caso contrario, no modifica el atributo y muestra: `"[ERROR] Category must be one of: PAINTING, SCULPTURE, MANUSCRIPT, TEXTILE, CERAMIC, OTHER."`.

- El método **`isValidConservationStatus`** devuelve `true` si el parámetro recibido coincide con alguno de los estados de conservación válidos, ignorando mayúsculas y minúsculas. En cualquier otro caso, devuelve `false`.

- El método **`isValueInArray`** recibe un texto y un *array* de valores admitidos. Devuelve `true` si el texto no es `null` y coincide con alguno de los elementos del *array*, sin distinguir mayúsculas y minúsculas; en cualquier otro caso, devuelve `false`. Los métodos `isValidCategory` e `isValidConservationStatus` deben reutilizarlo con sus respectivos valores admitidos.

- El método **`setConservationStatus`** asigna el estado en mayúsculas si es válido. En caso contrario, no modifica el atributo y muestra: `"[ERROR] Conservation status must be one of: EXCELLENT, GOOD, FAIR, POOR."`.

- El método **`setAcquisitionDate`** asigna la fecha de adquisición. El parámetro no puede ser `null` ni una fecha posterior a la actual. Si ya existe una fecha de última inspección, tampoco puede ser posterior a esta; ambas fechas pueden coincidir. Si no es válido, el método debe mostrar: `"[ERROR] Acquisition date cannot be null, after the last inspection date or in the future."`.



Pista: Investiga también sobre la clase `LocalDate` de la API de Java para obtener la fecha actual y también manipular fechas (p.ej. hacer comparaciones).

- El método **`setLastInspectionDate`** asigna la fecha de la última inspección. Para poder asignarla, la pieza debe tener ya una fecha de adquisición asignada. Además, el parámetro no puede ser `null`, anterior a la fecha de adquisición ni posterior a la fecha actual. Si no se cumple alguna de estas condiciones, el método no modifica el atributo y muestra: `"[ERROR] Last inspection date cannot be null, before the acquisition date or in the future."`.

- El método **`setAppraisedValue`** asigna el valor de tasación en euros, que debe ser estrictamente positivo. Si no es válido, no modifica el atributo y muestra: `"[ERROR] Appraised value must be greater than 0."`.

- Los métodos **`setFragile`** y **`setOnLoan`** asignan directamente los valores booleanos recibidos.

- El método **`isValidReferenceDate`** devuelve `true` cuando la fecha recibida no es `null` ni anterior a la última inspección. En cualquier otro caso, devuelve `false`.

- El método **`isValidLoanDays`** devuelve `true` cuando la duración recibida está comprendida entre `1` y `365` días, ambos incluidos.

- El método **`needsInspection`** recibe una fecha de referencia y devuelve `true` cuando han transcurrido al menos `12` meses desde la última inspección. Si la fecha no es válida, muestra: `"[ERROR] Reference date cannot be null or before the last inspection date."` y devuelve `false`.

- El método **`isHighValue`** devuelve `true` cuando el valor de tasación es igual o superior a `100.000` €.

- El método **`isHighRisk`** devuelve `true` cuando la pieza es frágil y se cumple al menos una de estas condiciones: (1) su estado es `FAIR` o `POOR`, o (2) necesita una nueva inspección. Si la fecha de referencia no es válida, muestra el mensaje correspondiente y devuelve `false`.

- El método **`canBeLoaned`** devuelve `true` únicamente cuando la pieza no está ya prestada, su estado es `EXCELLENT` o `GOOD` y no necesita una nueva inspección. Si la fecha de referencia no es válida, muestra el mensaje correspondiente y devuelve `false`.

- El método **`calculateInsuranceCost`** recibe la duración del préstamo. El coste inicial se calcula multiplicando el valor de tasación por el `0.01` % y por el número de días. A continuación, si la pieza es frágil, el importe se multiplica por `1.25`. Después,

si la pieza es de alto valor, el importe resultante se multiplica por `1.15`. Por tanto, cuando corresponden ambos recargos, se aplican sucesivamente, no sumando sus porcentajes sobre el coste inicial. Si la duración no está entre `1` y `365` días, ambos incluidos, muestra el mensaje `"[ERROR] Loan days must be between 1 and 365."` y devuelve `0.0`.

- El método **`isEligibleForLoan`** recibe una categoría solicitada, el valor máximo cubierto por el seguro, la duración del préstamo y la fecha de referencia. La categoría debe pertenecer a las categorías válidas, ignorando mayúsculas y minúsculas; el valor máximo asegurado debe ser estrictamente positivo; y la duración debe estar comprendida entre `1` y `365` días, ambos incluidos. Si alguno de estos tres datos no es válido, muestra el mensaje `"[ERROR] Invalid loan request data."` y devuelve `false`.

  A continuación, comprueba que la fecha de referencia no sea `null` ni anterior a la última inspección; si no es válida, muestra el mensaje `"[ERROR] Reference date cannot be null or before the last inspection date."` y devuelve `false`.

  Si todos los datos son válidos, el método devuelve `true` únicamente cuando se cumplen a la vez estas tres condiciones:

  - La categoría solicitada coincide con la categoría de la pieza, ignorando mayúsculas y minúsculas;
  - El valor de tasación de la pieza es menor o igual que el valor máximo asegurado;
  - La pieza puede prestarse en la fecha de referencia, según el método `canBeLoaned`.

  En cualquier otro caso, devuelve `false`. La duración del préstamo solo se valida; no interviene en la decisión.

- El método **`areLoanRequestsValid`** devuelve `true` cuando los *arrays* `requestedCategories`, `maxInsuredValues` y `loanDays` no son `null`, tienen la misma longitud y todos sus elementos cumplen las reglas de validación de categoría, valor máximo asegurado y duración descritas para `isEligibleForLoan`. Además, la fecha de referencia no puede ser `null` ni anterior a la última inspección. Se admiten tres *arrays* vacíos siempre que la fecha de referencia sea válida. En cualquier otro caso, devuelve `false`, sin mostrar mensajes por pantalla.

- El método **`countEligibleLoanRequests`** recorre los tres *arrays* y devuelve el número de solicitudes para las que la pieza resulta elegible. Si los datos no son válidos, muestra: `"[ERROR] Invalid input data for loan request calculation."` y devuelve `0`.

- El método **`calculateEligibleLoanRequestPercentage`** devuelve el porcentaje de solicitudes elegibles sobre el total. Si los *arrays* están vacíos, devuelve `0.0`. Si los datos no son válidos, muestra: `"[ERROR] Invalid input data for loan request calculation."` y devuelve `0.0`.

Un ejemplo de evaluación de solicitudes:
Supongamos una pieza de categoría `PAINTING`, tasada en `100 000 €`, que puede prestarse en la fecha de referencia. Los datos recibidos son:

Request	Requested category	Max insured value	Loan days	Valid data?	Eligible?
1	painting	120 000 €	10	Yes	Yes
2	SCULPTURE	150 000 €	20	Yes	No: category mismatch
3	PAINTING	80 000 €	15	Yes	No: insufficient insurance coverage
4	PAINTING	100 000 €	30	Yes	Yes

En este ejemplo:

- `isEligibleForLoan` devuelve, respectivamente, `true`, `false`, `false` y `true` al evaluar cada solicitud.
- `areLoanRequestsValid` devuelve `true`: todos los datos cumplen las reglas de validación, aunque algunas solicitudes no sean elegibles.
- `countEligibleLoanRequests` devuelve 2.
- `calculateEligibleLoanRequestPercentage` devuelve `50.0`: hay 2 solicitudes elegibles de un total de 4.

Si cambiamos la duración de la tercera solicitud de `15` a `0`, sus datos dejan de ser válidos. `areLoanRequestsValid` devuelve `false`; el contador y el cálculo del porcentaje muestran el mensaje de error correspondiente y devuelven `0` y `0.0`. La solicitud inválida no se omite: se rechaza el conjunto completo.

Con tres arrays vacíos y una fecha válida, `areLoanRequestsValid` devuelve `true`, el contador devuelve `0` y el porcentaje devuelve `0.0`, sin mostrar errores.



Al ejecutar los tests de JUnit, **no es necesario** disponer de un punto de entrada mediante un método `main`. El propio entorno de JUnit actúa como punto de entrada llamando directamente a los tests.



Nota: El enunciado incluye tests públicos para todos los métodos excepto **`isEligibleForLoan`**, **`countEligibleLoanRequests`** y **`calculateEligibleLoanRequestPercentage`**. Estos métodos se comprobarán mediante tests privados puntuables durante la corrección.

Debes crear tus propios tests para verificar su funcionamiento; no es necesario entregarlos.



Requisito mínimo para evaluar este ejercicio: La clase `MuseumArtifact` debe pasar todos los tests proporcionados.



Nota: El estudiante puede recibir una penalización de hasta **1 punto** de la nota obtenida en este ejercicio en función de la calidad del código proporcionado.

(5 puntos: 3.5 pts. tests proporcionados; 1.5 pts. tests privados usados durante la corrección: 0,3 puntos para `isEligibleForLoan`, 0,6 puntos para `countEligibleLoanRequests` y 0,6 puntos para `calculateEligibleLoanRequestPercentage`)

Ejercicio 2 (1 punto)

Antes de empezar, debes:

> Leer el apartado 3.12 de la guía de Java que habla sobre excepciones.
> Ver los videotutoriales sobre excepciones y cláusulas guarda que encontrarás en UOCoders.

Una vez realizada la tarea anterior, debes:

- Abrir el proyecto **`PAC2Ex2`** con IntelliJ.
- Copiar todo el *package* `edu.uoc.pac2` del `src/main/java` del Ejercicio 1 en el directorio `src/main/java` del proyecto **`PAC2Ex2`**.
- Modificar los métodos `setInventoryCode`, `setTitle`, `setCreator`, `setOriginCountry`, `setCategory`, `setConservationStatus`, `setAcquisitionDate`, `setLastInspectionDate` y `setAppraisedValue` para que, en vez de mostrar por pantalla el mensaje de error, lancen una `IllegalArgumentException` con el mismo mensaje sin el prefijo `[ERROR]`.
- Modificar los métodos `needsInspection`, `isHighRisk`, `canBeLoaned`, `calculateInsuranceCost` e `isEligibleForLoan` para que, cuando alguno de sus parámetros no sea válido, lancen una `Exception` con el mensaje correspondiente y no impriman ningún mensaje por pantalla.
- Modificar los métodos `countEligibleLoanRequests` y `calculateEligibleLoanRequestPercentage` para que, si los `arrays` o alguno de sus elementos no son válidos, o la fecha de referencia es null o anterior a la última inspección, lancen una `IllegalArgumentException` con el mensaje `"Invalid input data for loan request calculation."`, sin mostrar ningún mensaje por pantalla. La fecha de referencia debe validarse también cuando los tres arrays estén vacíos.
- Realizar las modificaciones auxiliares que consideres necesarias para trabajar con excepciones de forma eficiente, evitando repetir las mismas validaciones.



> **Requisito mínimo para evaluar este ejercicio:** La clase `MuseumArtifact` debe pasar los tests de todos los *setters*.

*(**1 punto: 0.6 puntos** correspondientes a los tests de los setters y **0.4 puntos** correspondientes a los tests de los métodos de negocio y de procesamiento de solicitudes)*

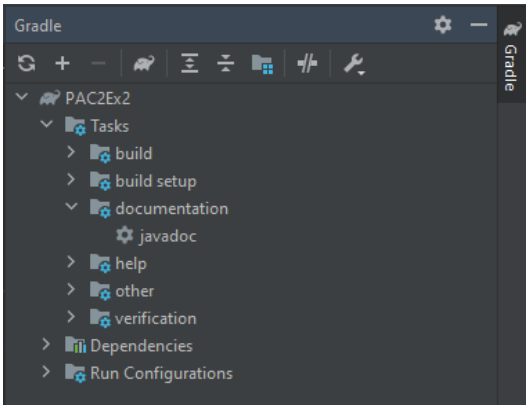
Ejercicio 3 (1 punto)

Antes de empezar, debes:

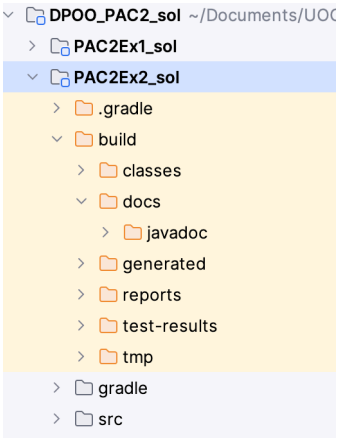
Leer el apartado 5.1 de la Guía de Java que habla de Javadoc.

En el proyecto `PAC2Ex2` documenta, con comentarios Javadoc, la clase `MuseumArtifact` que has codificado en el Ejercicio 2 de manera que describan los elementos de dicha clase: **la clase en sí, sus atributos y métodos (independientemente de su modificador de acceso)**. Los comentarios pueden ser en español o inglés, pero te recomendamos que sean en inglés.

Una vez documentada la clase, vamos a utilizar la tarea de Gradle `documentation→ javadoc` para generar la documentación (se podría generar usando directamente la opción de IntelliJ, pero ya que estamos usando Gradle, lo aprovechamos). Haz doble clic en esta tarea de Gradle (ver la siguiente imagen):



Esta tarea genera un directorio llamado `docs` dentro de la carpeta `build` del proyecto **`PAC2Ex2`** que incluye la documentación generada (dentro de la subcarpeta `javadoc`). La documentación que generes debe incluir la descripción de la clase `MuseumArtifact` junto con la descripción de todos sus atributos y métodos (independientemente de su modificador de acceso).



Si abres con un navegador web (puedes hacerlo con el menú contextual del botón derecho del mouse desde el propio IntelliJ) el fichero `index.html` de la carpeta `build/docs/javadoc`, podrás ver el enlace al javadoc de la clase `MuseumArtifact`. Al clicar en la clase, verás que tenemos toda la información pública de la misma, pero no obtenemos la información de los atributos (que son todos privados) ni de los métodos privados de dicha clase. Para obtener esta información en el javadoc, debemos añadir unas líneas en el fichero de configuración `build.gradle`:

```
javadoc {
   options.memberLevel = JavadocMemberLevel.PRIVATE
}
```

Después de añadir (y guardar) estas líneas al final del fichero, vuelve a ejecutar la tarea de Gradle `documentation → javadoc` para generar nuevamente la documentación, y verás que ahora sí tienes la información de los atributos y métodos privados.



Requisito mínimo para evaluar este ejercicio: Debe existir el directorio `build/docs/javadoc` con la documentación en formato web generada por la tarea `documentation → javadoc` de Gradle.

(1 punto: 0.5 puntos documentación; 0.5 calidad de los comentarios Javadoc)

Ejercicio 4 (3 puntos)

Antes de empezar, debes:

> Leer el apartado 4.1 de la Guía de Java que habla sobre la clase `String` y las clases relacionadas `StringBuilder` y `StringBuffer`.

Copia en tu *workspace* el proyecto **`PAC2Ex4`** que te facilitamos con el enunciado y ábrelo con IntelliJ. A continuación, debes codificar los `TODO` (del inglés, "*to do*", en español, "*por hacer*", o usando una traducción más libre: "*pendiente*") del *package* `edu.uoc.pac2` utilizando el potencial de las clases `String`, `StringBuffer` y `StringBuilder`.

En el package `edu.uoc.pac2`, encontrarás la clase `SampleLabelFormatter`. Un biobanco identifica cada muestra mediante un código y una etiqueta. Para detectar errores de transcripción, la etiqueta incluye un dígito de control. También es posible generar de una sola vez las etiquetas de un lote de muestras. Implementa los métodos de `SampleLabelFormatter` siguiendo las especificaciones que se indican a continuación.

Un código de muestra válido tiene exactamente el formato `BIO-NNNNNN`: el prefijo `BIO-` en mayúsculas seguido de **seis dígitos ASCII** (del `0` al `9`). No se aceptan espacios ni otros caracteres.

Los tipos de muestra válidos son `BLOOD`, `SALIVA`, `TISSUE` y `DNA`. Para reconocerlos se ignoran las mayúsculas y minúsculas, pero no se eliminan espacios.

El dígito de control se calcula multiplicando cada uno de los seis dígitos del código por su posición, numerada del `1` al `6` de izquierda a derecha. Se suman los seis productos y se toma el resto de dividir la suma entre `10`. Por ejemplo, para `BIO-123456`, la suma es `1×1 + 2×2 + 3×3 + 4×4 + 5×5 + 6×6 = 91`, por lo que el dígito de control es `1`.

Debes codificar los siguientes métodos siguiendo las indicaciones que te damos a continuación:

- **`isValidSampleCode`** recibe el parámetro `code` (`String`), que contiene el código identificador de una muestra. Devuelve un valor `boolean`: `true` si el código sigue exactamente el formato indicado y `false` si no lo cumple, incluido el caso en que `code` sea `null`.

  (0.4 puntos tests proporcionados)

- **`isValidSampleType`** recibe el parámetro `type` (`String`), que contiene el tipo de muestra. Devuelve un valor `boolean`: `true` si `type` coincide con uno de los cuatro tipos admitidos y `false` si no coincide, incluido el caso en que sea `null`.

  (0.3 puntos tests proporcionados)

- **`calculateCheckDigit`** recibe el parámetro `code` (`String`), que identifica una muestra, y devuelve un valor `int` con el dígito de control calculado según la fórmula descrita anteriormente. Si el código no es válido, lanza una excepción `IllegalArgumentException` con el mensaje `"Invalid sample code."`.



Pista: Recuerda que los dígitos ASCII representados como `char` permiten hacer operaciones aritméticas. Al restar `'0'` a un dígito obtienes su valor numérico; por ejemplo, `'6' - '0' = 6`.

(0.6 puntos tests proporcionados)

- **`buildLabel`** recibe `code` (`String`), que identifica la muestra, y `type` (`String`), que indica su tipo. Devuelve un `String` con la etiqueta en el formato `{CÓDIGO}|{TIPO}|{DÍGITO}`. El tipo debe aparecer en mayúsculas, aunque el parámetro se reciba en minúsculas o combinando ambas.

  Por ejemplo, al recibir `"BIO-123456"` y `"blood"`, devuelve `"BIO-123456|BLOOD|1"`. Si el código no es válido, lanza `IllegalArgumentException` con el mensaje `"Invalid sample code."`. Si el código es válido, pero el tipo no lo es, lanza la misma clase de excepción con el mensaje `"Invalid sample type."`. Debes construir la etiqueta mediante `StringBuilder`.



Pista: Utiliza `StringBuilder` para añadir sucesivamente el código, el tipo, los separadores y el dígito de control; es más eficiente que concatenar repetidamente objetos `String` con el operador `+`.

(0.9 puntos tests proporcionados)

- **`buildBatchLabels`** recibe `codes` (`String[]`), con los códigos de las muestras, y `types` (`String[]`), con sus tipos correspondientes. Los elementos en la misma posición de ambos *arrays* describen una misma muestra. Devuelve un `String` con una etiqueta por muestra, en el mismo orden y separadas por saltos de línea, sin añadir uno al final. Si ambos *arrays* están vacíos, devuelve `""`. Si alguno es `null`, sus longitudes son distintas o contienen algún elemento no válido, lanza `IllegalArgumentException` con el mensaje `"Invalid batch data."`.

  Por ejemplo, para los códigos `{"BIO-123456", "BIO-000000"}` y los tipos `{"blood", "DNA"}`, `buildBatchLabels` devuelve exactamente:

```
BIO-123456|BLOOD|1
BIO-000000|DNA|0
```



Nota: Para codificar este método, debes decidir si utilizar una variable de tipo `StringBuilder` o `StringBuffer`.

(0.8 puntos tests proporcionados)



Requisito mínimo para evaluar este ejercicio: El programa debe pasar todos los tests proporcionados para los métodos `isValidSampleCode` y `isValidSampleType`.



Nota: En la corrección se valorará también la calidad del código y se podrá descontar hasta **1 punto** del total. Se tendrá en cuenta, entre otros aspectos, la claridad del código, los nombres de las variables, el uso de constantes y la reutilización de métodos.

Formato y fecha de entrega

Tienes que entregar un fichero *.zip, cuyo nombre tiene que seguir este patrón: `loginUOC_PEC2.zip`. Por ejemplo: `dgarciaso_PEC2.zip`. Este fichero comprimido tiene que incluir los siguientes elementos:

- El proyecto de IntelliJ `PAC2Ex1` completado siguiendo las peticiones y especificaciones del Ejercicio 1.
- El proyecto de IntelliJ `PAC2Ex2` completado siguiendo las peticiones y especificaciones del Ejercicio 2, así como la documentación en formato Javadoc cumpliendo los requisitos indicados en el Ejercicio 3.
- El proyecto de IntelliJ `PAC2Ex4` completado siguiendo las peticiones y especificaciones del Ejercicio 4.

El último día para entregar esta PEC es el **26/10/2026** antes de las 23:59. Cualquier PEC entregada más tarde será considerada como no presentada.



No respetar el formato de entrega de un ejercicio supondrá la no corrección del mismo y, por tanto, una calificación de cero puntos en ese ejercicio.



El Equipo Docente podrá solicitar a cualquier estudiante que explique, justifique o modifique el código entregado, con el fin de verificar su autoría y garantizar que comprende adecuadamente la solución presentada. La imposibilidad de justificar de forma satisfactoria la solución podrá dar lugar a las actuaciones previstas en la normativa de integridad académica de la Universidad.

El medio utilizado para dicha justificación podrá ser diverso, en función de las circunstancias: entrevista oral síncrona mediante videoconferencia, comunicación por correo electrónico, prueba escrita síncrona o asíncrona, o cualquier otro procedimiento que permita verificar de manera adecuada la autoría y comprensión de la entrega.

Anexo: Problemas frecuentes

En este apartado se presentan algunos de los problemas más frecuentes observados durante la creación y ejecución de proyectos en IntelliJ relacionados con la PEC2. Si os encontráis con problemas no resueltos en este anexo, podéis utilizar el foro de la asignatura para resolverlos.

Del mismo modo que se mencionó en la PEC1, considerad aplicar las soluciones propuestas en este anexo a lo largo de la asignatura.

Espacios y caracteres especiales en la ruta del proyecto

En algunos sistemas operativos, especialmente con el uso de Gradle, que se usará a partir de la PEC2, pueden producirse errores de ejecución de código si la ruta absoluta del directorio contiene espacios o caracteres especiales. Por ejemplo, a la hora de ejecutar los tests de un proyecto Gradle con una ruta absoluta inválida, nos muestra el siguiente error:

```
Execution failed for task ':test'.
> There were failing tests. See the report at: file:///C:/git/UOC/DPOO/bad%20..%C2%BFpath%

* Try:
> Run with --scan to get full insights.
BUILD FAILED in 782ms
3 actionable tasks: 1 executed, 2 up-to-date
```

Por esta razón, es altamente recomendable asegurarse de que la ruta absoluta del directorio que almacena el proyecto de IntelliJ no contenga ningún espacio ni ningún carácter especial.

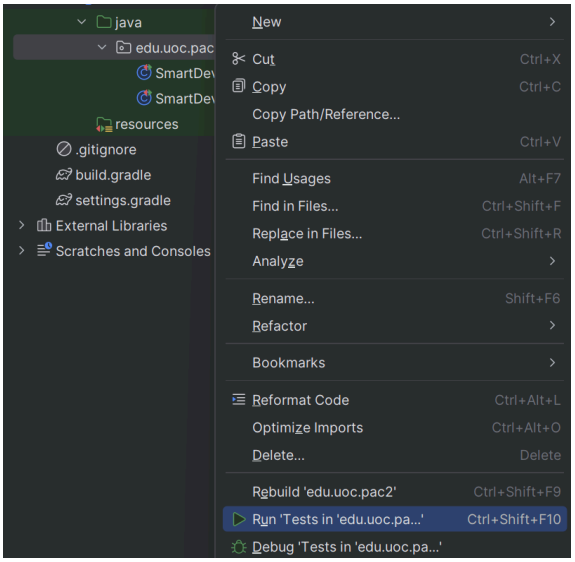
El IDE no encuentra ningún test para ejecutar

Notaréis que, con el uso de Gradle en IntelliJ, tendréis distintas maneras de ejecutar toda la colección de tests de las PECs. A veces, al ejecutar los tests mediante la tarea de Gradle del menú lateral, puede aparecer el siguiente mensaje:

```
Test events were not received
```

Este error se debe a ciertas incompatibilidades entre IntelliJ, Gradle y JUnit 5 en la ejecución de los tests.

En este caso, una alternativa que permite la ejecución de los tests incluso si se está trabajando con un proyecto Gradle es mediante una de las opciones del árbol de ficheros del proyecto.



Concretamente, podéis ejecutarlos mediante el menú contextual (botón derecho del ratón) del *package* que contiene los tests o directamente con el menú contextual del fichero que contiene los tests si solo se quiere ejecutar los tests de un archivo en concreto.

Limpiar el proyecto elimina el directorio *javadoc*

Antes de entregar un proyecto es recomendable eliminar todos los archivos temporales que se generan durante la ejecución del proyecto. Para ello, es posible utilizar la tarea de Gradle que encontraréis en `Tasks > build > clean`. No obstante, dependiendo de la ruta en la que se genere la documentación *javadoc* de un proyecto, es posible que esta también se elimine con la limpieza del proyecto. Esto ocurre cuando la carpeta de la documentación *javadoc* se genera dentro del directorio `build`.

En este caso, podéis modificar la ruta en la que se genera la documentación *javadoc* del proyecto para que no se elimine cuando se quiera limpiar el proyecto. Para ello, podéis hacerlo desde el menú superior con la opción `Tools > Generate JavaDoc > JavaDoc Options > Output directory`.

Si el ejercicio impide que modifiquéis la ruta o preferís no hacerlo, podéis mover la carpeta temporalmente fuera del directorio `build` para poder ejecutar la tarea `clean` o eliminar manualmente los archivos temporales. Como notaréis, esta última opción es menos elegante que la anterior, pero, en caso de necesidad, podría serviros como alternativa para solucionar el problema.